# Linux Kernel Programming

Linux Kernel programming

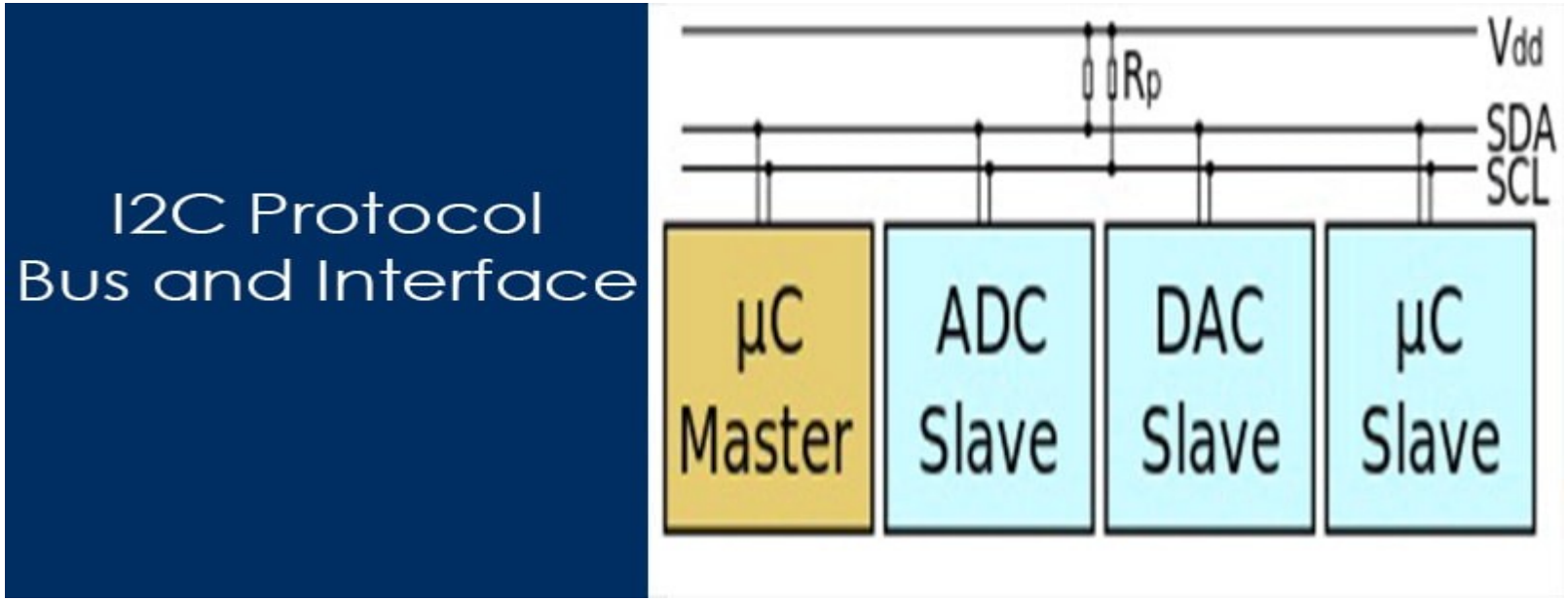
# Agenda

- I2C Userspace
- I2C Driver

# I2C Driver - Introduction

- I2C is a serial protocol for a two-wire interface to connect low-speed devices like microcontrollers, EEPROMs, A/D and D/A converters, I/O interfaces, and other similar peripherals in embedded systems. It was invented by Philips and now it is used by almost all major IC manufacturers.

- Each I2C slave device needs an address – they must still be obtained from NXP (formerly Philips semiconductors).



# I2C Driver - Introduction

- The I2C bus is popular because it is simple to use, there can be more than one master, only upper bus speed is defined and only two wires with pull-up resistors are needed to connect an almost unlimited number of I2C devices. I2C can use even slower microcontrollers with general-purpose I/O pins since they only need to generate correct Start and Stop conditions in addition to functions for reading and writing a byte.

- Each slave device has a unique address. Transfer from and to the master device is serial and it is split into 8-bit packets. All these simple requirements make it very simple to implement the I2C interface even with cheap microcontrollers that have no special I2C hardware controller. You only need 2 free I/O pins and few simple i2C routines to send and receive commands.

- I2C bus specification describes four operating speed categories for bidirectional data transfer:
  - Standard-mode(Sm) - a bit rate up to 100 kbit/s
  - Fast-mode(Fm) - a bit rate up to 400 kbit/s
  - Fast-mode Plus(Fm+) - a bit rate up to 1 Mbit/s
  - High-speed mode(Hs-mode) - a bit rate up to 3.4 Mbit/s
  - Ultra-fast mode(UFm) - a bit rate up to 5 Mbit/s

# I2C Driver - Introduction

- Basic I2C communication is using transfers of 8 bits or bytes. Each I2C slave device has a 7-bit address that needs to be unique on the bus. Some devices have fixed I2C addresses while others have few address lines which determine lower bits of the I2C address. This makes it very easy to have all I2C devices on the bus with a unique I2C address.

- There are also devices that have a 10-bit address as allowed by the specification.

- The 7-bit address represents bits 7 to 1 while bit 0 is used to signal reading from or writing to the device. If bit 0 (in the address byte) is set to 1 then the master device will read from the slave I2C device.

- The master device needs no address since it generates the clock (via SCL) and addresses individual I2C slave devices.





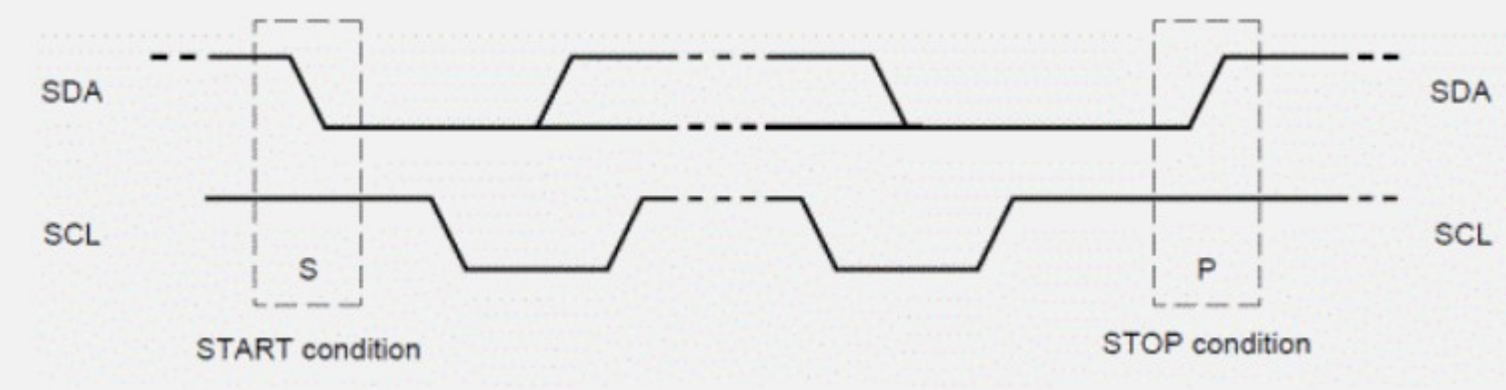

# I2C Driver - Introduction

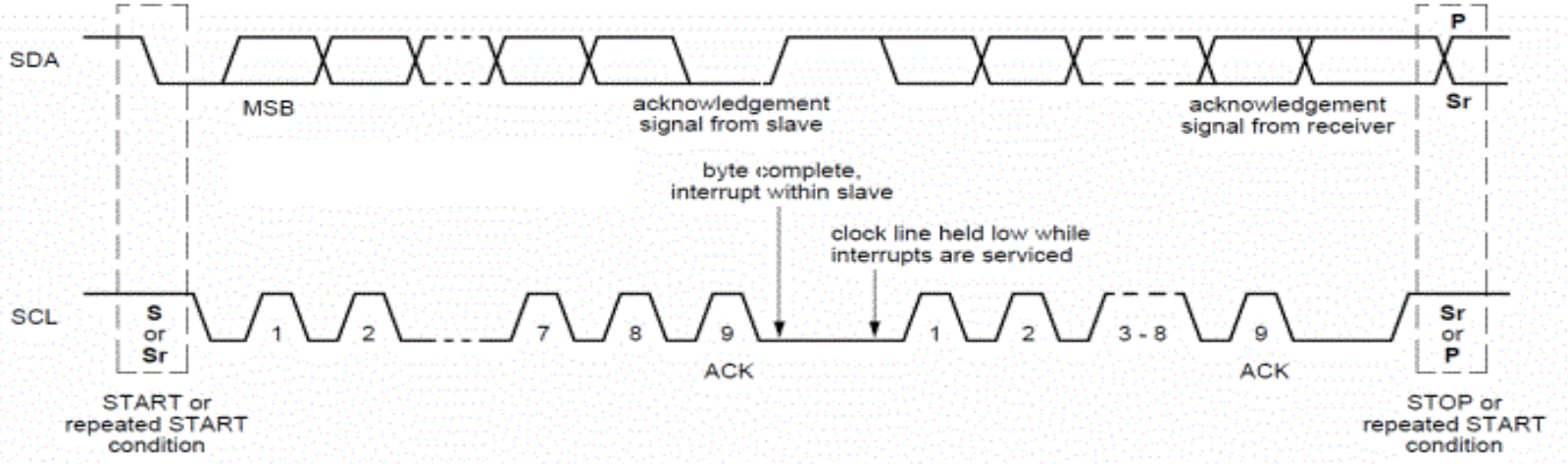
- Each I2C command initiated by the master device starts with a START condition and ends with a STOP condition. For both conditions, SCL has to be high. A high to low transition of SDA is considered as START and a low to high transition as STOP.

- After the Start condition, the bus is considered as busy and can be used by another master only after a Stop condition is detected. After the Start condition, the master can generate a repeated Start. This is equivalent to a normal Start and is usually followed by the slave I2C address.

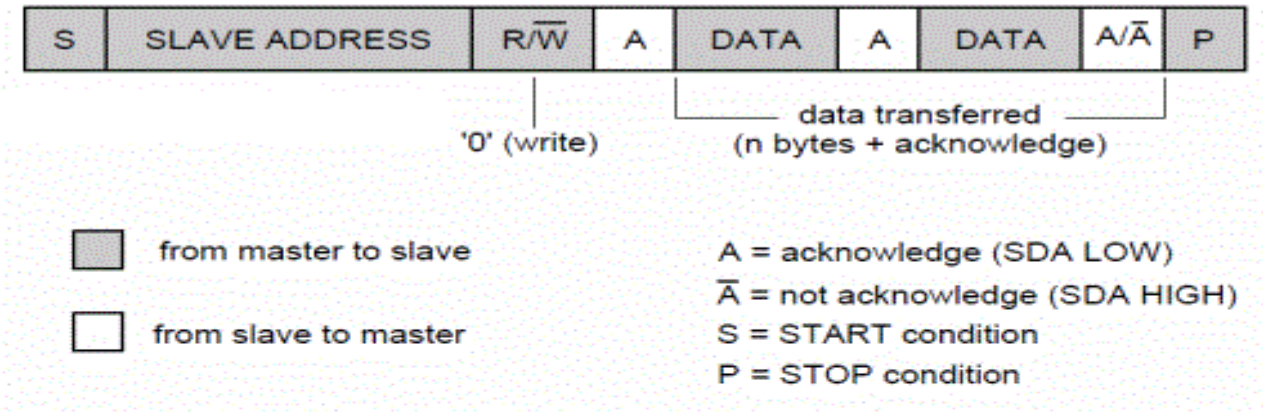
- Microcontrollers that have dedicated I2C hardware can easily detect bus changes and behave also as I2C slave devices. However, if the I2C communication is implemented in software, the bus signals must be sampled at least two times per clock cycle in order to detect necessary changes.

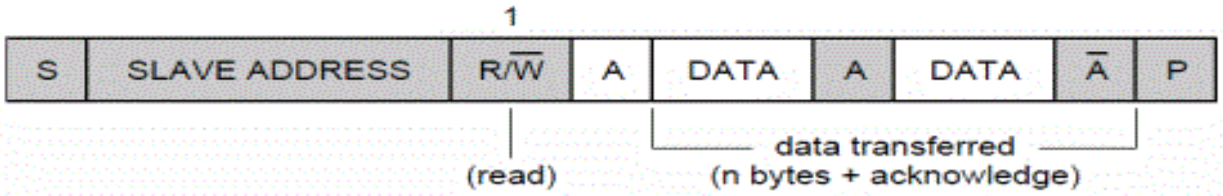
# I2C Driver - Introduction



# I2C Driver - Introduction



# I2C Driver - Introduction

| | I²C | SMBus |
|---|---|---|
| Timeout | No | Yes |
| Minimum Clock Speed | DC | 10kHz |
| Maximum Clock Speed | 100kHz (400kHz and 2MHz also available) | 100kHz |
| $V_{HIGH}$ | 0.7 × $V_{DD}$, 3.0V Fixed | 2.1V |
| $V_{LOW}$ | 0.3 × $V_{DD}$, 1.5V Fixed | 0.8V |
| Max I | 3mA | 350µA |
| Clock Nomenclature | SCL | SMBCLK |
| Data Nomenclature | SDA | SMBDAT |
| General Call | Yes | Yes |
| Alert# | No | Yes |

# I2C UserSpace

- We need to install the i2ctools package.

  **sudo apt-get update**

  **sudo apt-get install i2c-tools**

- You can test if i2ctools is working by listing all of the I²C devices connected to your Raspberry Pi.

  **sudo i2cdetect -y 1**

- i2cdetect will display a grid of numbers with the addresses of any I²C devices shown within the grid.

```
root@raspberrypi:/var# i2cdetect -y -a 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- 3c -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

# I2C UserSpace

- The I²C bus supports several bus speeds, typically 100KHz, 400KHz, 1MHz, 3.4MHz and 5.0MHz. You can set the bus speed by editing /boot/config.txt. Open the file with nano using the command:

  **sudo nano /boot/config.txt**

- Add the following text to the bottom of the file; The number is the frequency of the I²C bus in hertz:

  **dtparam=i2c_baudrate=400000**

- Save your changes, exit the nano editor and reboot

  **sudo reboot**

- Before the Raspberry Pi can communicate with an I²C device it needs to know the address assigned to that device. I2C devices typically use a 7-bit address which is sent at the start of each communication from the master to the slave device. 7 bits gives a total of 128 possible device addresses.

# I2C UserSpace

- Some devices will have their address pre-set by the manufacturer while others will have a small range of addresses that can be selected by the user. Each device on the bus must have a unique address, if two devices share the same address then there can be a conflict that stops the devices from functioning correctly.

- To find a list of the devices connected to the I²C bus on the Raspberry Pi you can use the following command:

  **sudo i2cdetect -y 1**

- I2cdetect returns a grid of possible addresses with each device displayed as shown in the screenshot below.

```
root@raspberrypi:/var# i2cdetect -y -a 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- 3c -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

# I2C UserSpace

- Available Commands in I2C Tools
  - **i2cdetect** Detect I2C chips connected to the bus.
  - **i2cdump** Examine and read I2C registers on a connected device.
  - **i2cget** Read from I2C/SMBus chip registers on a connected device.
  - **i2cset** Set I2C registers on a connected device with new data or values.
  - **i2ctransfer** Send user defined I2C messages in one transfer to a connected device.

# I2C UserSpace

**i2cdetect**

**i2cdetect [-y] [-a] [-q|-r] i2cbus [first last]**

**i2cdetect -F i2cbus**

**i2cdetect -V**

**i2cdetect –l**

**Description**

- i2cdetect is a program to scan an I2C bus for connected devices. The output is shown as a table with a list of the detected devices on the specified bus. The optional parameters first and last restrict the scanning range to the specified valued. (default: from 0x03 to 0x77).

# I2C UserSpace

**Options**

-y Disable interactive mode and perform operation directly. This is useful in scripts or when detecting devices.

-F List the functionality of the adapter.

-V Show the version and exit.

-l Show a list of installed I2C busses.

**Output**

Each cell in the output table will contain one of the following symbols:

- “UU” Probing of the address was skipped because the address is currently in use by a driver.
- An address number in hexadecimal format.
- “--" The address was probed but no response from a chip.

# I2C UserSpace

**Examples**

The examples below can be used on your Raspberry Pi with i2cdetect.

**i2cdetect -l**

List all detected devices on I2C bus 1 (default on Raspberry Pi) without user interaction.

**i2cdetect -y 1**

# I2C UserSpace

**i2cdump**

**i2cdump [-f] [-r first-last] [-y] [-a] i2cbus address [mode [bank [bankreg]]]**

**i2cdump -V**

**Description**

i2cdump is a helper program to read and examine registers on devices connected through the I2C bus.

**Options**

-y Disable interactive mode and perform operation directly. This is useful in scripts.

-f Force access to the device even if it is being used by another application or driver. Using this flag can cause unexpected results and can confuse the kernel driver using the device.

-V Show the version and exit.

# I2C UserSpace

- A minimum of two options must be provided with i2cdump. i2cbus is the number of the I2C bus to be scanned. address selects the I2C device address to be scanned on the bus and is an integer between 0x03 and 0x77.

**Examples**

Dump the content of an I2C device at 7-bit address 0x68 on bus 1 using the default read byte mode with a user confirmation:

**i2cdump -y 1 0x68**

Dump the content of an I2C device at 7-bit address 0x68 on bus 1 using I2C block read transactions without user confirmation:

**i2cdump -y 1 0x68 i**

# I2C UserSpace

**i2cget**

**i2cget [-f] [-y] [-a] i2cbus chip-address [data-address [mode]]**

**i2cget -V**

**Description**

i2cget is a helper program to read registers on devices connected through the I2C bus.

**Options**

-y Disable interactive mode and perform operation directly. This is useful in scripts.

-f Force access to the device even if it is being used by another application or driver. Using this flag can cause unexpected results and can confuse the kernel driver using the device.

-V Show the version and exit.

# I2C UserSpace

- A minimum of two options must be provided with i2cget. i2cbus is the number of the I2C bus to be scanned. address selects the I2C device address to be scanned on the bus and is an integer between 0x03 and 0x77.

- The data-address parameter specifies the address on the chip to read from and is an integer between 0x00 and 0xFF.

- The optional mode parameter is one of the letters b, w or c, corresponding to a read byte data, a read word data or a write byte/read byte transaction.

# I2C UserSpace

**Examples**

Read the content of an I2C device at 7-bit address 0x68 on bus 1 from an 8-bit register at 0x20 after user confirmation:

**i2cget 1 0x68 0x20**

Read the content of an I2C device at 7-bit address 0x68 on bus 1 from an 16-bit register at 0x20 after user confirmation:

**i2cget 1 0x68 0x20 w**

# I2C UserSpace

**i2cset**

**i2cset [-f] [-y] [-m mask] [-r] [-a] i2cbus chip-address data-address [value]  [mode]**

**i2cset –V**

**Description**

i2cset is a helper program to set registers on devices connected through the I2C bus.

**Options**

-y Disable interactive mode and perform operation directly. This is useful in scripts.

-f Force access to the device even if it is being used by another application or driver. Using this flag can cause unexpected results and can confuse the kernel driver using the device.

-V Show the version and exit.

-m mask The mask parameter, if specified, describes which bits of value will be actually written to data-address. Bits set to 1 in the mask are taken from value, while bits set to 0 will be read from data-address and thus preserved by the operation.

# I2C UserSpace

-r Read the value back after writing and compare with the value written.

- A minimum of three options must be provided with i2cset. i2cbus is the number of the I2C bus to be scanned.
- The data-address parameter selects the I2C device address to be scanned on the bus and is an integer between 0x03 and 0x77.
- The data-address parameter specifies the address on the chip to read from and is an integer between 0x00 and 0xFF.
- The value parameter if specified is the value to write to that location on the chip. If this parameter is omitted, then a short write is issued.
- The optional mode parameter, if specified, is one of the letters b, w, s, or i, corresponding to a write size of a single byte, a 16-bit word, a SMBus block write, or an I2C block write.

# I2C UserSpace

- For SMBus and I2C block writes, the write size is determined by the number of value parameters. Except for I2C block writes, a p can also be appended to the mode parameter to enable PEC. If the mode parameter is omitted, i2cset defaults to byte mode without PEC.

**Examples**

Write the value 0x50 to an 8-bit register 0x20 to a device on at 7-bit I2C address 0x68 on bus 1 after user confirmation:

**i2cset 1 0x68 0x20 0x50**

Write the value 0x5000 to a 16-bit register 0x20 to a device on at 7-bit I2C address 0x68 on bus 1 with no user confirmation:

**i2cset -y 1 0x68 0x20 0x5000 w**

# I2C UserSpace

**i2ctransfer**

**i2ctransfer [-f] [-y] [-v] [-a] i2cbus desc [data] [desc [data]]**

**i2ctransfer -V**

**Description**

- i2ctransfer is a program to create I2C messages and send them combined as one transfer to devices connected through the I2C bus.

- For read messages, the contents of the received buffers are printed to stdout, one line per read message.

**Options**

-y Disable interactive mode and perform operation directly. This is useful in scripts.

-f Force access to the device even if it is being used by another application or driver. Using this flag can cause unexpected results and can confuse the kernel driver using the device.

# I2C UserSpace

-V Show the version and exit.

-v Enable verbose output. This will print information about all messages sent, both for read and write.

**Arguments**

The first parameter i2cbus contains the number of the I2C bus to be used. On a Raspberry Pi this should be 1.

**EXAMPLE**

- On bus 0, from an EEPROM at address 0x50, read 8 bytes from offset 0x64 (first message writes one byte to set the memory pointer to 0x64, second message reads from the same chip):

**i2ctransfer 0 w1@0x50 0x64 r8**

# I2C and SMBus

- I²C (pronounce: I squared C and written I2C in the kernel documentation) is a protocol developed by Philips. It is a slow two-wire protocol (variable speed, up to 400 kHz), with a high speed extension (3.4 MHz). It provides an inexpensive bus for connecting many types of devices with infrequent or low bandwidth communications needs. I2C is widely used with embedded systems. Some systems use variants that don't meet branding requirements, and so are not advertised as being I2C but come under different names, e.g. TWI (Two Wire Interface), IIC.

- SMBus (System Management Bus) is based on the I2C protocol, and is mostly a subset of I2C protocols and signaling. Many I2C devices will work on an SMBus, but some SMBus protocols add semantics beyond what is required to achieve I2C branding. Modern PC mainboards rely on SMBus. The most common devices connected through SMBus are RAM modules configured using I2C EEPROMs, and hardware monitoring chips.

- Because the SMBus is mostly a subset of the generalized I2C bus, we can use its protocols on many I2C systems. However, there are systems that don't meet both SMBus and I2C electrical constraints; and others which can't implement all the common SMBus protocol semantics or messages.

# I2C and SMBus

- Using the terminology from the official documentation, the I2C bus connects one or more master chips and one or more slave chips.

- A master chip is a node that starts communications with slaves. In the Linux kernel implementation it is called an adapter or bus. Adapter drivers are in the drivers/i2c/busses/ subdirectory.

- An algorithm contains general code that can be used to implement a whole class of I2C adapters. Each specific adapter driver either depends on an algorithm driver in the drivers/i2c/algos/ subdirectory, or includes its own implementation.

- A slave chip is a node that responds to communications when addressed by the master. In Linux it is called a client. Client drivers are kept in a directory specific to the feature they provide, for example drivers/media/gpio/ for GPIO expanders and drivers/media/i2c/ for video-related chips.

- For the example configuration in figure, you will need a driver for your I2C adapter, and drivers for your I2C devices (usually one driver for each device).

# I2C Protocol

| | |
|---|---|
| S | Start condition |
| P | Stop condition |
| Rd/Wr (1 bit) | Read/Write bit. Rd equals 1, Wr equals 0. |
| A, NA (1 bit) | Acknowledge (ACK) and Not Acknowledge (NACK) bit |
| Addr (7 bits) | I2C 7 bit address. Note that this can be expanded to get a 10 bit I2C address. |
| Data (8 bits) | A plain data byte. |
| [..] | Data sent by I2C device, as opposed to data sent by the host adapter. |

## Simple send transaction

Implemented by **`i2c_master_send()`**:

```
S Addr Wr [A] Data [A] Data [A] ... [A] Data [A] P
```

## Simple receive transaction

Implemented by **`i2c_master_recv()`**:

```
S Addr Rd [A] [Data] A [Data] A ... A [Data] NA P
```

## Combined transactions

Implemented by **`i2c_transfer()`**.

They are just like the above transactions, but instead of a stop condition P a start condition S is sent and the transaction continues. An example of a byte read, followed by a byte write:

```
S Addr Rd [A] [Data] NA S Addr Wr [A] Data [A] P
```

# I2C Protocol

| | |
|---|---|
| S | Start condition |
| Sr | Repeated start condition, used to switch from write to read mode. |
| P | Stop condition |
| Rd/Wr (1 bit) | Read/Write bit. Rd equals 1, Wr equals 0. |
| A, NA (1 bit) | Acknowledge (ACK) and Not Acknowledge (NACK) bit |
| Addr (7 bits) | I2C 7 bit address. Note that this can be expanded to get a 10 bit I2C address. |
| Comm (8 bits) | Command byte, a data byte which often selects a register on the device. |
| Data (8 bits) | A plain data byte. DataLow and DataHigh represent the low and high byte of a 16 bit word. |
| Count (8 bits) | A data byte containing the length of a block operation. |
| [..] | Data sent by I2C device, as opposed to data sent by the host adapter. |

## SMBus Quick Command

This sends a single bit to the device, at the place of the Rd/Wr bit:

```
S Addr Rd/Wr [A] P
```

Functionality flag: I2C_FUNC_SMBUS_QUICK

# I2C Protocol

## SMBus Receive Byte

Implemented by **`i2c_smbus_read_byte()`**

This reads a single byte from a device, without specifying a device register. Some devices are so simple that this interface is enough; for others, it is a shorthand if you want to read the same register as in the previous SMBus command:

```
S Addr Rd [A] [Data] NA P
```

Functionality flag: I2C_FUNC_SMBUS_READ_BYTE

## SMBus Send Byte

Implemented by **`i2c_smbus_write_byte()`**

This operation is the reverse of Receive Byte: it sends a single byte to a device. See Receive Byte for more information.

```
S Addr Wr [A] Data [A] P
```

Functionality flag: I2C_FUNC_SMBUS_WRITE_BYTE

# I2C Protocol

## SMBus Read Byte

Implemented by **`i2c_smbus_read_byte_data()`**

This reads a single byte from a device, from a designated register. The register is specified through the Comm byte:

```
S Addr Wr [A] Comm [A] Sr Addr Rd [A] [Data] NA P
```

Functionality flag: I2C_FUNC_SMBUS_READ_BYTE_DATA

## SMBus Read Word

Implemented by **`i2c_smbus_read_word_data()`**

This operation is very like Read Byte; again, data is read from a device, from a designated register that is specified through the Comm byte. But this time, the data is a complete word (16 bits):

```
S Addr Wr [A] Comm [A] Sr Addr Rd [A] [DataLow] A [DataHigh] NA P
```

Functionality flag: I2C_FUNC_SMBUS_READ_WORD_DATA

Note the convenience function i2c_smbus_read_word_swapped() is available for reads where the two data bytes are the other way around (not SMBus compliant, but very popular.)

# I2C Protocol

## SMBus Write Byte

Implemented by **`i2c_smbus_write_byte_data()`**

This writes a single byte to a device, to a designated register. The register is specified through the Comm byte. This is the opposite of the Read Byte operation.

```
S Addr Wr [A] Comm [A] Data [A] P
```

Functionality flag: I2C_FUNC_SMBUS_WRITE_BYTE_DATA

## SMBus Write Word

Implemented by **`i2c_smbus_write_word_data()`**

This is the opposite of the Read Word operation. 16 bits of data are written to a device, to the designated register that is specified through the Comm byte:

```
S Addr Wr [A] Comm [A] DataLow [A] DataHigh [A] P
```

Functionality flag: I2C_FUNC_SMBUS_WRITE_WORD_DATA

Note the convenience function i2c_smbus_write_word_swapped() is available for writes where the two data bytes are the other way around (not SMBus compliant, but very popular.)

# I2C Protocol

## SMBus Process Call

This command selects a device register (through the Comm byte), sends 16 bits of data to it, and reads 16 bits of data in return:

```
S Addr Wr [A] Comm [A] DataLow [A] DataHigh [A]
                           Sr Addr Rd [A] [DataLow] A [DataHigh] NA P
```

Functionality flag: I2C_FUNC_SMBUS_PROC_CALL

## SMBus Block Read

Implemented by **`i2c_smbus_read_block_data()`**

This command reads a block of up to 32 bytes from a device, from a designated register that is specified through the Comm byte. The amount of data is specified by the device in the Count byte.

```
S Addr Wr [A] Comm [A]
          Sr Addr Rd [A] [Count] A [Data] A [Data] A ... A [Data] NA P
```

Functionality flag: I2C_FUNC_SMBUS_READ_BLOCK_DATA

# I2C Protocol

## SMBus Block Write

Implemented by **`i2c_smbus_write_block_data()`**

The opposite of the Block Read command, this writes up to 32 bytes to a device, to a designated register that is specified through the Comm byte. The amount of data is specified in the Count byte.

```
S Addr Wr [A] Comm [A] Count [A] Data [A] Data [A] ... [A] Data [A] P
```

Functionality flag: I2C_FUNC_SMBUS_WRITE_BLOCK_DATA

## SMBus Block Write - Block Read Process Call

SMBus Block Write - Block Read Process Call was introduced in Revision 2.0 of the specification.

This command selects a device register (through the Comm byte), sends 1 to 31 bytes of data to it, and reads 1 to 31 bytes of data in return:

```
S Addr Wr [A] Comm [A] Count [A] Data [A] ...
                           Sr Addr Rd [A] [Count] A [Data] ... A P
```

Functionality flag: I2C_FUNC_SMBUS_BLOCK_PROC_CALL

# I2C Protocol

## SMBus Host Notify

This command is sent from a SMBus device acting as a master to the SMBus host acting as a slave. It is the same form as Write Word, with the command code replaced by the alerting device's address.

```
[S] [HostAddr] [Wr] A [DevAddr] A [DataLow] A [DataHigh] A [P]
```

This is implemented in the following way in the Linux kernel:

- I2C bus drivers which support SMBus Host Notify should report I2C_FUNC_SMBUS_HOST_NOTIFY.
- I2C bus drivers trigger SMBus Host Notify by a call to **`i2c_handle_smbus_host_notify()`**.
- I2C drivers for devices which can trigger SMBus Host Notify will have client->irq assigned to a Host Notify IRQ if no one else specified another.

There is currently no way to retrieve the data parameter from the client.

## Packet Error Checking (PEC)

Packet Error Checking was introduced in Revision 1.1 of the specification.

PEC adds a CRC-8 error-checking byte to transfers using it, immediately before the terminating STOP.

## Address Resolution Protocol (ARP)

The Address Resolution Protocol was introduced in Revision 2.0 of the specification. It is a higher-layer protocol which uses the messages above.

ARP adds device enumeration and dynamic address assignment to the protocol. All ARP communications use slave address 0x61 and require PEC checksums.

# I2C Protocol

## SMBus Alert

SMBus Alert was introduced in Revision 1.0 of the specification.

The SMBus alert protocol allows several SMBus slave devices to share a single interrupt pin on the SMBus master, while still allowing the master to know which slave triggered the interrupt.

This is implemented the following way in the Linux kernel:

- I2C bus drivers which support SMBus alert should call **`i2c_new_smbus_alert_device()`** to install SMBus alert support.
- I2C drivers for devices which can trigger SMBus alerts should implement the optional alert() callback.

## I2C Block Transactions

The following I2C block transactions are similar to the SMBus Block Read and Write operations, except these do not have a Count byte. They are supported by the SMBus layer and are described here for completeness, but they are *NOT* defined by the SMBus specification.

I2C block transactions do not limit the number of bytes transferred but the SMBus layer places a limit of 32 bytes.

## I2C Block Read

Implemented by i2c_smbus_read_i2c_block_data()

This command reads a block of bytes from a device, from a designated register that is specified through the Comm byte:

```
S Addr Wr [A] Comm [A]
          Sr Addr Rd [A] [Data] A [Data] A ... A [Data] NA P
```

Functionality flag: I2C_FUNC_SMBUS_READ_I2C_BLOCK

# I2C Protocol

## I2C Block Write

Implemented by i2c_smbus_write_i2c_block_data()

The opposite of the Block Read command, this writes bytes to a device, to a designated register that is specified through the Comm byte. Note that command lengths of 0, 2, or more bytes are supported as they are indistinguishable from data.

```
S Addr Wr [A] Comm [A] Data [A] Data [A] ... [A] Data [A] P
```

Functionality flag: I2C_FUNC_SMBUS_WRITE_I2C_BLOCK

# How to instantiate I2C devices

- Unlike PCI or USB devices, I2C devices are not enumerated at the hardware level. Instead, the software must know which devices are connected on each I2C bus segment, and what address these devices are using. For this reason, the kernel code must instantiate I2C devices explicitly. There are several ways to achieve this, depending on the context and requirements.

**Method 1: Declare the I2C devices statically**

- This method is appropriate when the I2C bus is a system bus as is the case for many embedded systems. On such systems, each I2C bus has a number which is known in advance. It is thus possible to pre-declare the I2C devices which live on this bus. This information is provided to the kernel in a different way on different architectures: device tree, ACPI or board files. When the I2C bus in question is registered, the I2C devices will be instantiated automatically by i2c-core. The devices will be automatically unbound and destroyed when the I2C bus they sit on goes away (if ever).

# How to instantiate I2C devices

**Declare the I2C devices via devicetree**

- On platforms using devicetree, the declaration of I2C devices is done in sub nodes of the master controller.

Example:

```
i2c1: i2c@400a0000 {
        /* ... master properties skipped ... */
        clock-frequency = <100000>;

        flash@50 {
                compatible = "atmel,24c256";
                reg = <0x50>;
        };

        pca9532: gpio@60 {
                compatible = "nxp,pca9532";
                gpio-controller;
                #gpio-cells = <2>;
                reg = <0x60>;
        };
};
```

- Here, two devices are attached to the bus using a speed of 100kHz. For additional properties which might be needed to set up the device, please refer to its devicetree documentation in Documentation/devicetree/bindings/.

# How to instantiate I2C devices

**Declare the I2C devices via ACPI**

- ACPI can also describe I2C devices. There is special documentation for this which is currently located at ACPI Based Device Enumeration.

- The slaves behind I2C bus controller only need to add the ACPI IDs like with the platform and SPI drivers. The I2C core automatically enumerates any slave devices behind the controller device once the adapter is registered.

- Below is an example of how to add ACPI support to the existing mpu3050 input driver:

```
static const struct acpi_device_id mpu3050_acpi_match[] = {
        { "MPU3050", 0 },
        { }
};
MODULE_DEVICE_TABLE(acpi, mpu3050_acpi_match);

static struct i2c_driver mpu3050_i2c_driver = {
        .driver = {
                .name   = "mpu3050",
                .pm     = &mpu3050_pm,
                .of_match_table = mpu3050_of_match,
                .acpi_match_table = mpu3050_acpi_match,
        },
        .probe          = mpu3050_probe,
        .remove         = mpu3050_remove,
        .id_table       = mpu3050_ids,
};
module_i2c_driver(mpu3050_i2c_driver);
```

# How to instantiate I2C devices

**Declare the I2C devices in board files**

- In many embedded architectures, devicetree has replaced the old hardware description based on board files, but the latter are still used in old code. Instantiating I2C devices via board files is done with an array of struct i2c_board_info which is registered by calling i2c_register_board_info().

Example (from omap2 h4):

```
static struct i2c_board_info h4_i2c_board_info[] __initdata = {
	{
		I2C_BOARD_INFO("isp1301_omap", 0x2d),
		.irq		= OMAP_GPIO_IRQ(125),
	},
	{	/* EEPROM on mainboard */
		I2C_BOARD_INFO("24c01", 0x52),
		.platform_data	= &m24c01,
	},
	{	/* EEPROM on cpu card */
		I2C_BOARD_INFO("24c01", 0x57),
		.platform_data	= &m24c01,
	},
};

static void __init omap_h4_init(void)
{
	(...)
	i2c_register_board_info(1, h4_i2c_board_info,
			ARRAY_SIZE(h4_i2c_board_info));
	(...)
}
```

- The above code declares 3 devices on I2C bus 1, including their respective addresses and custom data needed by their drivers.

# How to instantiate I2C devices

**Method 2: Instantiate the devices explicitly**

- This method is appropriate when a larger device uses an I2C bus for internal communication. A typical case is TV adapters. These can have a tuner, a video decoder, an audio decoder, etc. usually connected to the main chip by the means of an I2C bus. You won't know the number of the I2C bus in advance, so the method 1 described above can't be used. Instead, you can instantiate your I2C devices explicitly. This is done by filling a struct i2c_board_info and calling i2c_new_client_device().

Example (from the sfe4001 network driver):

```
static struct i2c_board_info sfe4001_hwmon_info = {
      I2C_BOARD_INFO("max6647", 0x4e),
};

int sfe4001_init(struct efx_nic *efx)
{
      (...)
      efx->board_info.hwmon_client =
              i2c_new_client_device(&efx->i2c_adap, &sfe4001_hwmon_info);

      (...)
}
```

- The above code instantiates 1 I2C device on the I2C bus which is on the network adapter in question.

# How to instantiate I2C devices

- A variant of this is when you don't know for sure if an I2C device is present or not (for example for an optional feature which is not present on cheap variants of a board but you have no way to tell them apart), or it may have different addresses from one board to the next (manufacturer changing its design without notice). In this case, you can call i2c_new_scanned_device() instead of i2c_new_client_device().

Example (from the nxp OHCI driver):

```
static const unsigned short normal_i2c[] = { 0x2c, 0x2d, I2C_CLIENT_END };

static int usb_hcd_nxp_probe(struct platform_device *pdev)
{
      (...)
      struct i2c_adapter *i2c_adap;
      struct i2c_board_info i2c_info;

      (...)
      i2c_adap = i2c_get_adapter(2);
      memset(&i2c_info, 0, sizeof(struct i2c_board_info));
      strscpy(i2c_info.type, "isp1301_nxp", sizeof(i2c_info.type));
      isp1301_i2c_client = i2c_new_scanned_device(i2c_adap, &i2c_info,
                                                  normal_i2c, NULL);
      i2c_put_adapter(i2c_adap);
      (...)
}
```

- The above code instantiates up to 1 I2C device on the I2C bus which is on the OHCI adapter in question. It first tries at address 0x2c, if nothing is found there it tries address 0x2d, and if still nothing is found, it simply gives up.The driver which instantiated the I2C device is responsible for destroying it on cleanup. This is done by calling i2c_unregister_device() on the pointer that was earlier returned by i2c_new_client_device() or i2c_new_scanned_device().

# How to instantiate I2C devices

**Method 3: Probe an I2C bus for certain devices**

- Sometimes you do not have enough information about an I2C device, not even to call i2c_new_scanned_device(). The typical case is hardware monitoring chips on PC mainboards. There are several dozen models, which can live at 25 different addresses. Given the huge number of mainboards out there, it is next to impossible to build an exhaustive list of the hardware monitoring chips being used. Fortunately, most of these chips have manufacturer and device ID registers, so they can be identified by probing.

- In that case, I2C devices are neither declared nor instantiated explicitly. Instead, i2c-core will probe for such devices as soon as their drivers are loaded, and if any is found, an I2C device will be instantiated automatically. In order to prevent any misbehavior of this mechanism, the following restrictions apply:
  - The I2C device driver must implement the detect() method, which identifies a supported device by reading from arbitrary registers.
  - Only buses which are likely to have a supported device and agree to be probed, will be probed. For example this avoids probing for hardware monitoring chips on a TV adapter.

# How to instantiate I2C devices

Example: See lm90_driver and lm90_detect() in drivers/hwmon/lm90.c

- I2C devices instantiated as a result of such a successful probe will be destroyed automatically when the driver which detected them is removed, or when the underlying I2C bus is itself destroyed, whichever happens first.

- Those of you familiar with the I2C subsystem of 2.4 kernels and early 2.6 kernels will find out that this method 3 is essentially similar to what was done there. Two significant differences are:
  - Probing is only one way to instantiate I2C devices now, while it was the only way back then. Where possible, methods 1 and 2 should be preferred. Method 3 should only be used when there is no other way, as it can have undesirable side effects.
  - I2C buses must now explicitly say which I2C driver classes can probe them (by the means of the class bitfield), while all I2C buses were probed by default back then. The default is an empty class which means that no probing happens. The purpose of the class bitfield is to limit the aforementioned undesirable side effects.

- Once again, method 3 should be avoided wherever possible. Explicit device instantiation (methods 1 and 2) is much preferred for it is safer and faster.

# How to instantiate I2C devices

- While this interface should only be used when in-kernel device declaration can't be done, there is a variety of cases where it can be helpful:
  - The I2C driver usually detects devices (method 3 above) but the bus segment your device lives on doesn't have the proper class bit set and thus detection doesn't trigger.
  - The I2C driver usually detects devices, but your device lives at an unexpected address.
  - The I2C driver usually detects devices, but your device is not detected, either because the detection routine is too strict, or because your device is not officially supported yet but you know it is compatible.
  - You are developing a driver on a test board, where you soldered the I2C device yourself.

- This interface is a replacement for the force_* module parameters some I2C drivers implement. Being implemented in i2c-core rather than in each device driver individually, it is much more efficient, and also has the advantage that you do not have to reload the driver to change a setting. You can also instantiate the device before the driver is loaded or even available, and you don't need to know what driver the device needs.

# How to instantiate I2C devices

**Method 4: Instantiate from user-space**

- In general, the kernel should know which I2C devices are connected and what addresses they live at. However, in certain cases, it does not, so a sysfs interface was added to let the user provide the information. This interface is made of 2 attribute files which are created in every I2C bus directory: new_device and delete_device. Both files are write only and you must write the right parameters to them in order to properly instantiate, respectively delete, an I2C device.

- File new_device takes 2 parameters: the name of the I2C device (a string) and the address of the I2C device (a number, typically expressed in hexadecimal starting with 0x, but can also be expressed in decimal.)

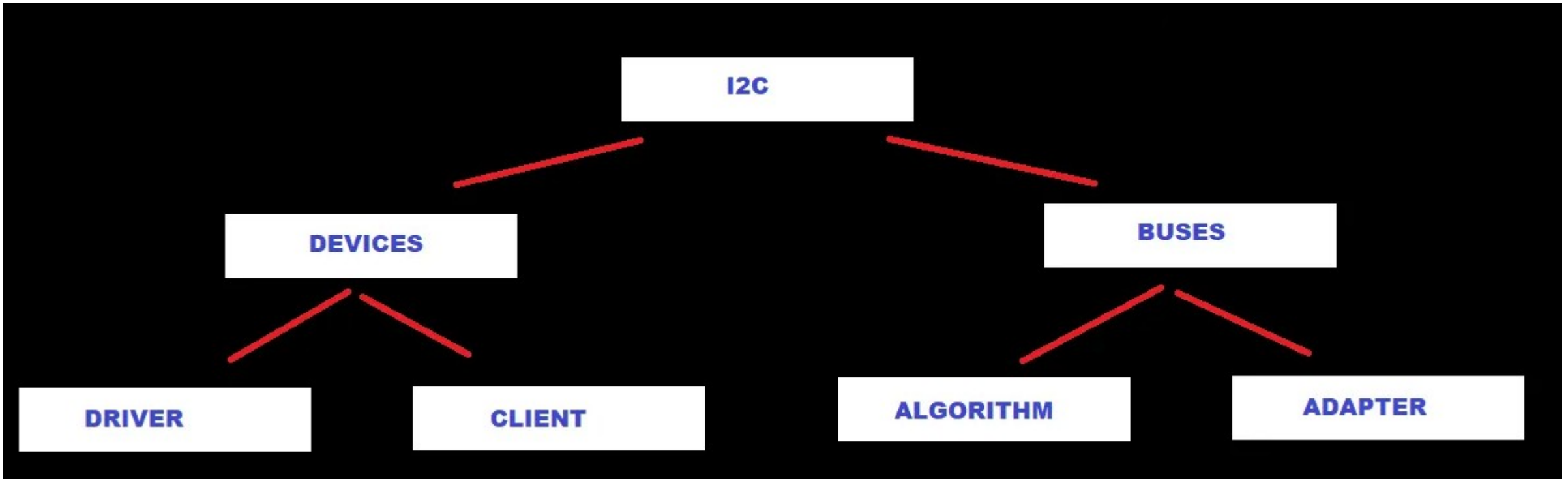
- File delete_device takes a single parameter: the address of the I2C device. As no two devices can live at the same address on a given I2C segment, the address is sufficient to uniquely identify the device to be deleted.

Example:

```
# echo eeprom 0x50 > /sys/bus/i2c/devices/i2c-3/new_device
```

# I2C Driver - Linux

- The kernel divided the I2C subsystem by Buses and Devices. The Buses are again divided into Algorithms and Adapters. The devices are again divided into Drivers and Clients. The below image will give you some understandings.



# I2C Driver - Linux

## Algorithm

An Algorithm driver contains a general code that can be used for a whole class of I2C adapters.

## Adapters

An Adapter effectively represents a bus – it is used to tie up a particular I2C with an algorithm and bus number. Each specific adapter driver either depends on one algorithm driver or includes its own implementation.

## Clients

A Client represents a chip (slave) on the I2C.

## Drivers

This is the driver that we are writing for the client.

# I2C Driver - Linux

- Usually, Driver and Client are more closely integrated than Algorithm and Adapter.

- So, you will need a driver for your I2C bus, and drivers for your I2C devices (usually one driver for each device).

# I2C Driver - Linux

- Steps that involve while writing the I2C device driver are given below.
  - Get the I2C adapter.
  - Create the i2c_board_info structure and create a device using that.
  - Create the i2c_device_id for slave device and register that.
  - Create the i2c_driver structure and add that to the I2C subsystem.
  - Now the driver is ready. We can transfer the data between master and slave.
  - Once we are done, then remove the device.

# I2C Driver - Linux

## Get the I2C adapter

- In raspberry Pi, the i2c-1 bus is available already. You can check that using the command ls -al /sys/bus/i2c/devices/. So we will use the below API to get the adapter structure of this I2C bus.

**struct i2c_adapter *i2c_get_adapter(int nr);**

**Where,**

**nr** – I2C bus number. In our case (Raspberry Pi), it should be 1.

- It returns the struct i2c_adapter.

# I2C Driver - Linux

**Create the board info**

- Once you get the adapter structure, then create the board info and using that board info, create the device.

- Just create the i2c_board_info structure and assign required members of that.

```
struct i2c_board_info {
  char type[I2C_NAME_SIZE];
  unsigned short flags;
  unsigned short addr;
  void * platform_data;
  struct dev_archdata * archdata;
  struct device_node * of_node;
  struct fwnode_handle * fwnode;
  int irq;
};
```

# I2C Driver - Linux

**where,**

**type[I2C_NAME_SIZE]** – chip type, to initialize i2c_client.name

**flags** – to initialize i2c_client.flags

**addr** – stored in i2c_client.addr

**platform_data** – stored in i2c_client.dev.platform_data

**archdata** – copied into i2c_client.dev.archdata

**of_node** – pointer to OpenFirmware device node

**fwnode** – device node supplied by the platform firmware

**irq** – stored in i2c_client.irq

- we can use I2C_BOARD_INFO macro to initialize the essential fields of struct i2c_board_info.

# I2C Driver - Linux

**I2C_BOARD_INFO ( dev_type, dev_addr);**

**where,**

**dev_type** – identifies the device type

**dev_addr** – the device's address on the bus

# I2C Driver - Linux

## Create Device

- We can instantiate the device from the I2C bus.

**struct i2c_client * i2c_new_client_device ( struct i2c_adapter * adap, struct i2c_board_info const * info);**

where,

***adap** – Adapter structure that we got from i2c_get_adapter()

***info** – Board info structure that we have created

- This will return the i2c_client structure. We can use this client for our future transfers.

# I2C Driver - Linux

```
#define I2C_BUS_AVAILABLE      ( 1 )            // I2C Bus available in our Raspberry Pi
#define SLAVE_DEVICE_NAME   ( “TEST_OLED" )          // Device and Driver Name
#define SSD1306_SLAVE_ADDR  (      0x3C )          // SSD1306 OLED Slave Address
static struct i2c_adapter *test_i2c_adapter     = NULL;  // I2C Adapter Structure
static struct i2c_client  *test_i2c_client_oled = NULL;  // I2C Cient Structure (In our case it is OLED)
/*
** I2C Board Info strucutre
*/
static struct i2c_board_info oled_i2c_board_info = {
        I2C_BOARD_INFO(SLAVE_DEVICE_NAME, SSD1306_SLAVE_ADDR)
    };
```

# I2C Driver - Linux

```
/*
** Module Init function
*/
static int __init test_driver_init(void)
{
    ....

    test_i2c_adapter     = i2c_get_adapter(I2C_BUS_AVAILABLE);
    test_i2c_client_oled = i2c_new_device(test_i2c_adapter, &oled_i2c_board_info);

    ....
    return 0;
}
```

# I2C Driver - Linux

- Now we have to create the i2c driver for our slave. In order to do that, you have to create the device id and i2c_driver. Then add that driver to the I2C subsystem.

# I2C Driver - Linux

**Create the device id**

- Just create the structure i2c_device_id and initialize the necessary members.

```
struct i2c_device_id {
  char name[I2C_NAME_SIZE];
  kernel_ulong_t driver_data;
};
```

**where,**

**name** – Slave name

**driver_data** – Data private to the driver (This data will be passed to the respective driver)

- After this, call MODULE_DEVICE_TABLE(i2c, my_id_table) in order to expose the driver along with its I2C device table IDs to userspace.

# I2C Driver - Linux

**Create the i2c_driver**

```
struct i2c_driver {
  unsigned int class;
  int (* attach_adapter) (struct i2c_adapter *);
  int (* probe) (struct i2c_client *, const struct i2c_device_id *);
  int (* remove) (struct i2c_client *);
  void (* shutdown) (struct i2c_client *);
  void (* alert) (struct i2c_client *, unsigned int data);
  int (* command) (struct i2c_client *client, unsigned int cmd, void *arg);
  struct device_driver driver;
  const struct i2c_device_id * id_table;
  int (* detect) (struct i2c_client *, struct i2c_board_info *);
  const unsigned short * address_list;
  struct list_head clients;
};
```

# I2C Driver - Linux

**Where,**

**class** – What kind of i2c device we instantiate (for detect)

**attach_adapter** – Callback for bus addition (deprecated)

**probe** – Callback for device binding

**remove** – Callback for device unbinding

**shutdown** – Callback for device shutdown

**alert** – Alert callback, for example for the SMBus alert protocol

**command** – Callback for bus-wide signaling (optional)

**driver** – Device driver model driver

**id_table** – List of I2C devices supported by this driver

**detect** – Callback for device detection

**address_list** – The I2C addresses to probe (for detect)

**clients** – List of detected clients we created (for i2c-core use only)

# I2C Driver - Linux

**Add the I2C driver to the I2C subsystem**

- Now we have the i2c_driver structure. So we can add this structure to the I2C subsystem using the below API.

  **i2c_add_driver(struct i2c_driver *i2c_drive);**

  **Where,**

  **i2c_drive** – The i2c_driver structure that we have created.

- During the call to i2c_add_driver to register the I2C driver, all the I2C devices will be traversed. Once matched, the probe function of the driver will be executed.

# I2C Driver - Linux

- You can remove the driver using i2c_del_driver(struct i2c_driver *i2c_drive).

# I2C Driver - Linux

```
/*
** This function getting called when the slave has been found
** Note : This will be called only once when we load the driver.
*/
static int test_oled_probe(struct i2c_client *client,
                           const struct i2c_device_id *id)
{
    ...

    return 0;
}
```

# I2C Driver - Linux

```
/*
** This function getting called when the slave has been removed
** Note : This will be called only once when we unload the driver.
*/
static int test_oled_remove(struct i2c_client *client)
{
    ...
    return 0;
}
```

# I2C Driver - Linux

```
/*
** Structure that has slave device id
*/
static const struct i2c_device_id test_oled_id[] = {
        { SLAVE_DEVICE_NAME, 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, test_oled_id);
```

# I2C Driver - Linux

```
/*
** I2C driver Structure that has to be added to linux
*/
static struct i2c_driver test_oled_driver = {
        .driver = {
            .name   = SLAVE_DEVICE_NAME,
            .owner  = THIS_MODULE,
        },
        .probe          = test_oled_probe,
        .remove          = test_oled_remove,
        .id_table       = test_oled_id,
};
```

# I2C Driver - Linux

```
/*
** Module Init function
*/
static int __init test_driver_init(void)
{
    ...
    i2c_add_driver(&test_oled_driver);
    ...
    return 0;
}
```

# I2C Driver - Linux

**i2c_master_send**

This API issue a single I2C message in the master transmit mode.

**int i2c_master_send ( const struct i2c_client * client, const char * buf, int count);**

**Where,**

**client** – Handle to the slave device

**buf** – Data that will be written to the slave

**count** – How many bytes to write, must be less than 64k since msg length is u16

- It returns negative errno, or else the number of bytes written.

# I2C Driver - Linux

**i2c_smbus_write_byte**

- This API is used to send one byte to the slave.

**s32 i2c_smbus_write_byte ( const struct i2c_client * client, u8 value);**

**Where,**

**client** – Handle to the slave device

**value** – Byte to be sent

- It returning negative errno else zero on success.

# I2C Driver - Linux

**i2c_smbus_write_byte_data**

**s32 i2c_smbus_write_byte_data ( const struct i2c_client * client, u8 command, u8 value);**

**Where,**

**client** – Handle to the slave device

**command** – Byte interpreted by slave

**value** – Byte being written

- It returning negative errno else zero on success.

# I2C Driver - Linux

**i2c_smbus_write_word_data**

**s32 i2c_smbus_write_word_data ( const struct i2c_client * client, u8 command, u16 value);**

**Where,**

**client** – Handle to the slave device

**command** – Byte interpreted by slave

**value** – 16-bit “word” being written

- It returning negative errno else zero on success.

# I2C Driver - Linux

**i2c_smbus_write_block_data**

**s32 i2c_smbus_write_block_data ( const struct i2c_client * client, u8 command, u8 length, const u8 * values);**

**Where,**

**client** – Handle to the slave device

**command** – Byte interpreted by slave

**length** – Size of the data block; SMBus allows at most 32 bytes

**values** – Byte array which will be written

- It returning negative errno else zero on success.

# I2C Driver - Linux

**i2c_master_recv**

- This API issue a single I2C message in master receive mode.

**int i2c_master_recv ( const struct i2c_client * client, const char * buf, int count);**

**Where,**

**client** – Handle to the slave device

**buf** – Data that will be read from the slave

**count** – How many bytes to read, must be less than 64k since msg length is u16

- It returns negative errno, or else the number of bytes reads.

# I2C Driver - Linux

**i2c_smbus_read_byte**

**s32 i2c_smbus_read_byte ( const struct i2c_client * client);**

**Where,**

**client** – Handle to the slave device

- It is returning negative errno else the byte received from the device.

# I2C Driver - Linux

**i2c_smbus_read_byte_data**

**s32 i2c_smbus_read_byte_data ( const struct i2c_client * client, u8 command);**

**Where,**

**client** – Handle to the slave device

**command** – Byte interpreted by slave

- It is returning negative errno else a data byte received from the device.

# I2C Driver - Linux

**i2c_smbus_read_word_data**

**s32 i2c_smbus_read_word_data ( const struct i2c_client * client, u8 command);**

**Where,**

**client** – Handle to the slave device

**command** – Byte interpreted by slave

- This returns negative errno else a 16-bit unsigned “word” received from the device.

# I2C Driver - Linux

**i2c_smbus_read_block_data**

**s32 i2c_smbus_read_block_data ( const struct i2c_client * client, u8 command, u8 * values);**

**Where,**

**client** – Handle to the slave device

**command** – Byte interpreted by slave

**values** – Byte array into which data will be read; big enough to hold the data returned by the slave. SMBus allows at most 32 bytes.

- This returns negative errno else the number of data bytes in the slave's response.

# I2C Driver - Linux

**i2c_transfer**

- If you want to send multiple I2C messages then you can use the below-given API.

**int i2c_transfer ( struct i2c_adapter * adap, struct i2c_msg * msgs, int num);**

**Where,**

**adap** – Handle to the I2C bus

**msgs** – One or more messages to execute before STOP is issued to terminate the operation; each message begins with a START.

**num** – Number of messages to be executed.

- It returns negative errno, else the number of messages executed.

# I2C Driver – Bus (Adapter)

- There are two structures that you need to use in order to write the i2c bus driver in the Linux kernel.
  - Algorithm Structure
  - Adapter Structure

# I2C Driver – Bus (Adapter)

**Algorithm Structure**

- This structure represents the I2C transfer method.

```
struct i2c_algorithm {
  int (* master_xfer) (struct i2c_adapter *adap, struct i2c_msg *msgs,int num);
  int (* smbus_xfer) (struct i2c_adapter *adap, u16 addr,unsigned short flags, char read_write,u8 command, int size, union i2c_smbus_data *data);
  u32 (* functionality) (struct i2c_adapter *);
#if IS_ENABLED(CONFIG_I2C_SLAVE)
  int (* reg_slave) (struct i2c_client *client);
  int (* unreg_slave) (struct i2c_client *client);
#endif
};
```

# I2C Driver – Bus (Adapter)

**Where,**

**master_xfer** — Issue a set of i2c transactions to the given I2C adapter defined by the msgs array, with num messages available to transfer via the adapter specified by adap. This function will be called whenever we call I2C read-write APIs from the client driver.

**smbus_xfer** — Issue SMBus transactions to the given I2C adapter. If this is not present, then the bus layer will try and convert the SMBus calls into I2C transfers instead. This function will be called whenever we call SMBus read-write APIs from the client driver.

**functionality** — Return the flags that this algorithm/adapter pair supports from the I2C_FUNC_* flags.

**reg_slave** — Register given client to I2C slave mode of this adapter

**unreg_slave** — Unregister given client from I2C slave mode of this adapter

- The return codes from the master_xfer field should indicate the type of error code that occurred during the transfer, as documented in the kernel Documentation file Documentation/i2c/fault-codes.

# I2C Driver – Bus (Adapter)

**Adapter Structure**

- This structure is used to identify a physical i2c bus along with the access algorithms necessary to access it.

```
struct i2c_adapter {
    struct module *owner;
    unsigned int class;
    const struct i2c_algorithm *algo;
    void *algo_data;
    struct rt_mutex bus_lock;
    int timeout;
    int retries;
    struct device dev;
    int nr;
```

# I2C Driver – Bus (Adapter)

```
    char name[48];
    struct completion dev_released;
    struct mutex userspace_clients_lock;
    struct list_head userspace_clients;
};
```

**Where,**

***owner** — Owner of the module(usually set this to THIS_MODULE).

**class** — the type of I2C class devices that this driver supports. Usually, this is set to any one of the I2C_CLASS_* based on our need.

***algo** — a pointer to the struct i2c_algorithm structure

**bus_lock** — Mutex lock.

**timeout** — Timeout in jiffies.

**retries** — Number of retries.

**dev** — the adapter device.

# I2C Driver – Bus (Adapter)

**nr** — bus number which you want to create. This will be applicable only for i2c_add_numbered_adapter().

**char name[I2C_NAME_SIZE]** — I2C bus driver name. This value shows up in the sysfs filename associated with this I2C adapter.

- After you create the two structures, then we have to add the adapter to the i2c subsystem.

# I2C Driver – Bus (Adapter)

**Add the adapter to the subsystem**

**i2c_add_adapter**

- This API is used to register the adapter to the subsystem. But this will assign the dynamic bus number.

  **int i2c_add_adapter (struct i2c_adapter * adapter);**

  **where,**

  **adapter** – the adapter to add

- It returns zero when a new bus number was allocated and stored in adap->nr, and the specified adapter became available for clients. Otherwise, a negative errno value is returned

# I2C Driver – Bus (Adapter)

**i2c_add_numbered_adapter**

- This API is used to register the adapter to the subsystem. But it assigns the number that we asked for if only it is available. We have to initialize the member called nr in the i2c_adapter structure before calling this.

  **int i2c_add_numbered_adapter ( struct i2c_adapter * adap);**

  **Where,**

  adap – the adapter to register (with adap->nr initialized)

- This returns zero when the specified adapter is available for clients using the bus number provided in adap->nr. Otherwise, a negative errno value is returned.

# I2C Driver – Bus (Adapter)

```
/*
** I2C adapter Structure
*/
static struct i2c_adapter test_i2c_adapter = {
    .owner  = THIS_MODULE,
    .class  = I2C_CLASS_HWMON,//| I2C_CLASS_SPD,
    .algo   = &test_i2c_algorithm,
    .name   = ADAPTER_NAME,
    .nr     = 5,
};
```

# I2C Driver – Bus (Adapter)

```
/*
** Module Init function
*/
static int __init test_driver_init(void)
{
    int ret = -1;

    //create bus driver as i2c-5 if it is available
    ret = i2c_add_numbered_adapter(&test_i2c_adapter);

    pr_info("Bus Driver Added!!!\n");
    return ret;
}
```

- If bus number 5 is not used by anyone, then it will assign that bus number to the client. Otherwise, it will return a negative number.

# I2C Driver – Bus (Adapter)

**Delete the adapter from the subsystem**

This API is used to unregister the adapter from the subsystem.

**void i2c_del_adapter ( struct i2c_adapter * adap);**

**Where,**

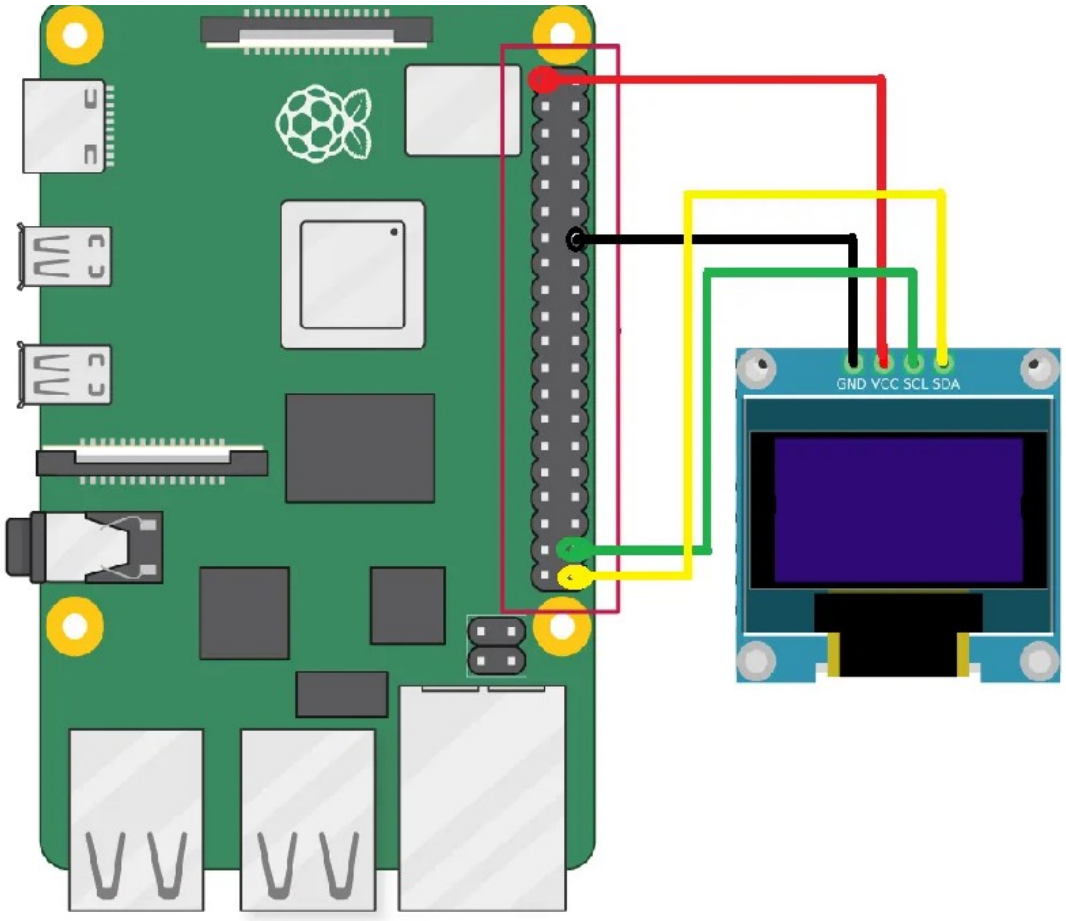
**adap** – the adapter being unregistered

# I2C Driver (Bit Banging Method) – GPIO (Method1)

**Connection Diagram**

SCL – GPIO 20

SDA – GPIO 21



# I2C Driver (Bit Banging Method) – GPIO (Method2)

- we have implemented everything using Bit banging right from the START condition, STOP condition, Read ACK/NACK, etc. But we have not taken care of advanced I2C operations like arbitration, clock stretching, etc. So even if we implement that we have to test it fully.

- So in this tutorial, we are going to use i2c-gpio driver where all the START condition, STOP condition, ACK/NACK, Arbitration, clock stretching, etc are implemented (bit-banging) and it has been tested fully as it is part of the Linux kernel. So we will see how to use that. Let's get started.

# I2C Driver (Bit Banging Method) – GPIO (Method2)

**struct i2c_algo_bit_data**

```
/*
 * This struct contains the hw-dependent functions of bit-style adapters to
 * manipulate the line states, and to init any hw-specific features. This is
 * only used if you have more than one hw-type of adapter running.
 */
```

```
struct i2c_algo_bit_data {
    void *data;     /* private data for lowlevel routines */
    void (*setsda) (void *data, int state);
    void (*setscl) (void *data, int state);
    int  (*getsda) (void *data);
    int  (*getscl) (void *data);
    int  (*pre_xfer)  (struct i2c_adapter *);
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
    void (*post_xfer) (struct i2c_adapter *);
    /* local settings */
    int udelay;     /* half clock cycle time in us,
                minimum 2 us for fast-mode I2C,
                minimum 5 us for standard-mode I2C and SMBus,
                maximum 50 us for SMBus */
    int timeout;        /* in jiffies */
    bool can_do_atomic; /* callbacks don't sleep, we can be atomic */
};
```

- If you see the structure, setscl, getscl, setsda, getsda are present. We have to give our callback function to these members. For example, If the i2c-gpio driver wants to send the START condition, it needs to set the SCL/SDA GPIO to High/Low. But it doesn't know which GPIO we are using. That's why they have made this as a callback.

# I2C Driver (Bit Banging Method) – GPIO (Method2)

**data** – private data (Optional)

**pre_xfer** – Function that will be called before i2C-gpio starts the I2C transfer (Optional)

**post_xfer** – Function that will be called after i2C-gpio finishes the I2C transfer (Optional)

**delay** – delay in uS

**timeout** – Timeout in jiffies

**can_do_atomic** – This will tells callback can sleep or not (Optional)

- When you are using this method, you have to add this structure to the i2c_adapter structure's algo_data member. Make sure that you have requested the GPIO before accessing it.

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
#define ADAPTER_NAME     “TEST_I2C_ADAPTER"
#define SCL_GPIO  20    //GPIO act as SCL
#define SDA_GPIO  21    //GPIO act as SDA
/*
** Function to read the SCL GPIO
*/
static int TEST_I2C_Read_SCL(void *data)
{
  gpio_direction_input(SCL_GPIO);
  return gpio_get_value(SCL_GPIO);
}
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
/*
** Function to read the SDA GPIO
*/
static int TEST_I2C_Read_SDA(void *data)
{
  gpio_direction_input(SDA_GPIO);
  return gpio_get_value(SDA_GPIO);
}
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
/*
** Function to set the SCL GPIO
*/
static void TEST_I2C_Set_SCL(void *data, int state)
{
  gpio_direction_output(SCL_GPIO, state);
  gpio_set_value(SCL_GPIO, state);
}
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
/*
** Function to set the SDA GPIO
*/
static void TEST_I2C_Set_SDA(void *data, int state)
{
  gpio_direction_output(SDA_GPIO, state);
  gpio_set_value(SDA_GPIO, state);
}
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
/*
** I2C bit algorithm Structure
*/
struct i2c_algo_bit_data test_bit_data = {
  .setsda = TEST_I2C_Set_SDA,
  .setscl = TEST _I2C_Set_SCL,
  .getscl = TEST _I2C_Read_SCL,
  .getsda = TEST _I2C_Read_SDA,
  .udelay = 5,
  .timeout = 100,       /* 100 ms */
};
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

```
/*
** I2C adapter Structure
*/
static struct i2c_adapter testg_i2c_adapter = {
  .owner      = THIS_MODULE,
  .class      = I2C_CLASS_HWMON | I2C_CLASS_SPD,
  .name       = ADAPTER_NAME,
  .algo_data  = &test_bit_data,
  .nr         = 5,
};
```

# I2C Driver (Bit Banging Method) – GPIO (Method2)

**i2c_bit_add_bus**

- This API is used to register the adapter to the subsystem. But this will assign the dynamic bus number.

  **int i2c_bit_add_bus (struct i2c_adapter * adapter);**

  where,

  **adapter –** the adapter to add

- It returns zero when a new bus number was allocated and stored in adap->nr, and the specified adapter became available for clients. Otherwise, a negative errno value is returned

# I2C Driver (Bit Banging Method) – GPIO (Method2)

**i2c_bit_add_numbered_bus**

- This API is used to register the adapter to the subsystem. But it assigns the number that we asked for if only it is available. We have to initialize the member called nr in the i2c_adapter structure before calling this.

  **int i2c_bit_add_numbered_bus ( struct i2c_adapter * adap);**

  **Where,**

  **adap** – the adapter to register (with adap->nr initialized)

- This returns zero when the specified adapter is available for clients using the bus number provided in adap->nr. Otherwise, a negative errno value is returned.

# Thank you